CHRISTOPHER NEWPORT UNIVERSITY · SCHOOL OF ENGINEERING & COMPUTING

GPT Assistant

Training Pipeline

Understanding how large language models evolve from raw data to intelligent conversational agents

Keith Perkins · School of Engineering & Computing

OVERVIEW

From data to intelligence

Building sophisticated AI assistants like GPT requires a carefully orchestrated multi-stage process. Each stage builds upon the previous one, progressively refining the model's capabilities from basic pattern recognition to nuanced, context-aware interactions.

STAGE 1

Pretraining

Learning language fundamentals from massive text corpora

STAGE 2

Supervised Fine-tuning

Teaching assistant-like behavior through curated examples

STAGE 3

Reward Modeling

Capturing human preferences through comparative rankings

STAGE 4

Reinforcement Learning

Optimizing responses for maximum human alignment

SEE THE DIFFERENCE

One prompt, three models

PROMPT **What is the capital of France?**

Illustrative outputs

BASE MODEL · AFTER STAGE 1

```
What is the capital of
Germany?
What is the capital of
Italy?
What is the capital of
Spain?
```

It continues the document. To a base model, a question looks like the start of a quiz, so it writes more questions.

SFT MODEL · AFTER STAGE 2

```
The capital of France is
Paris.
```

It answers. Fine-tuning on assistant conversations taught it the question-then-answer format.

RLHF MODEL · AFTER STAGE 4

```
Paris is the capital of
France. It is also the
country's largest city and
the seat of its national
government.
```

Tuned toward what raters prefer: correct, complete, and pitched at the right length.

Building the foundation of language understanding

Pretraining transforms raw internet text into a sophisticated language model through next-token prediction. The model processes trillions of tokens, learning grammar, facts, reasoning patterns, and world knowledge by continuously predicting what comes next in sequences.

Dataset characteristics: Massive scale (trillions of tokens) from diverse sources, heavily filtered and deduplicated for quality

Infrastructure requirements: Thousands to tens of thousands of GPUs training for weeks to months

Output: A powerful base model capable of generating coherent text but not yet optimized for conversational interactions

16K

GPUS AT FRONTIER SCALE

Llama 3 405B (2024) trained on 16,384 H100s

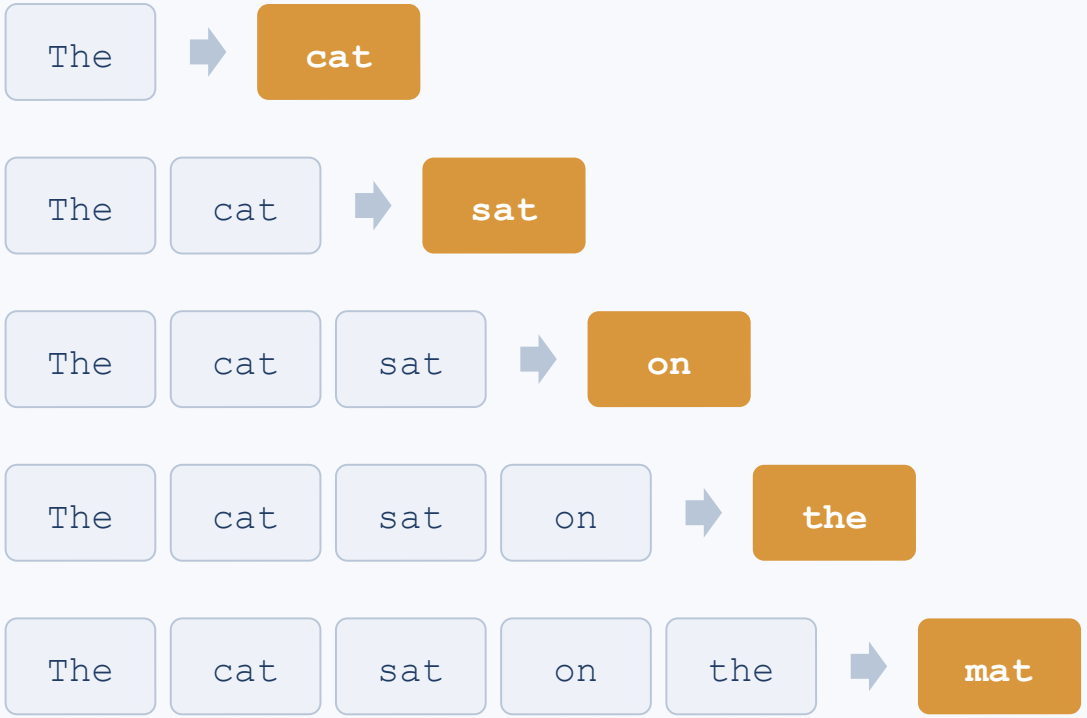
15T

TRAINING TOKENS

Llama 3: roughly 15 trillion tokens of filtered text

Next-token prediction, one token at a time

ONE SENTENCE BECOMES MANY TRAINING EXAMPLES



THE OBJECTIVE

At every position the model outputs a probability for every token in its vocabulary (often 100K+ tokens).

Cross entropy Loss
= −log P(correct next token)

averaged over every position, across trillions of tokens.

No labels needed: the text is its own answer key.

How much compute does pretraining take?

A rule of thumb lets you estimate training cost on the back of an envelope. Example: a 7-billion-parameter model trained on 2 trillion tokens (the size of Llama 2 7B).

STEP 1

Count the FLOPs

FLOPs ≈ 6 × N × D

N = parameters, D = training tokens

6 = 2 FLOPs forward + 4 backward, per parameter per token

STEP 2

Plug in the numbers

$6 \times (7 \times 10^{9}) \times (2 \times 10^{12})$

$= 8.4 \times 10^{22}$ FLOPs

STEP 3

Convert to GPU time

A100: 312 TFLOP/s peak, ~40% achieved ≈ 125 TFLOP/s

$8.4 \times 10^{22} \div 1.25 \times 10^{14} \approx 6.7 \times 10^{8}$ GPU-seconds

≈ 187,000 GPU-hours

≈ 1,000 GPUs for 8 days

Check: Meta reported **184,320 A100 GPU-hours** for Llama 2 7B, within 2% of our estimate.

Cheat sheet: GPUs and the recipe

NVIDIA A100

312
Peak TFLOP/s
BF16, dense: the spec sheet number

≈ 125
Typical TFLOP/s
~40% of peak in real training

80 GB
Memory
per GPU

NVIDIA H100

989
Peak TFLOP/s
BF16, dense: the spec sheet number

≈ 400
Typical TFLOP/s
~40% of peak in real training

80 GB
Memory
per GPU

THE RECIPE

1. **FLOPs = 6 × N × D**
   N = parameters, D = training tokens
2. **GPU-seconds = FLOPs ÷ typical FLOP/s**
   A100: 1.25×10^{14} · H100: 4×10^{14}
3. **GPU-hours = GPU-seconds ÷ 3,600**
   The cost unit: what cloud providers bill
4. **Days = GPU-hours ÷ (number of GPUs × 24)**
   Wall-clock time, if the GPUs run in parallel

Units: B = 10^9 · T (trillion) = 10^{12} · 1 TFLOP/s = 10^{12} FLOPs per second

Your turn: how long does pretraining take?

PROBLEM 1

8B model, 1T tokens

Parameters: 8 billion

Tokens: 1 trillion

Hardware: 1,024 A100s

Find: GPU-hours and days

PROBLEM 2

Same model, 15T tokens

Parameters: 8 billion

Tokens: 15 trillion (what Llama 3 8B used)

Hardware: A100s or H100s

Find: GPU-hours on each GPU type, and days on 1,024 H100s

PROBLEM 3

Bigger model: 70B, 2T tokens

Parameters: 70 billion

Tokens: 2 trillion

Hardware: 2,048 A100s

Find: GPU-hours and days

All three are Stage 1 (pretraining). Use the cheat sheet: typical A100 = 1.25×10^{14} FLOP/s, H100 = 4×10^{14} FLOP/s.

8B model, 1T tokens, 1,024 A100s

STEP 1

Count the FLOPs

$6 \times (8 \times 10^9) \times (1 \times 10^{12})$

$= 4.8 \times 10^{22}$ FLOPs

STEP 2

GPU time

$4.8 \times 10^{22} \div 1.25 \times 10^{14}$

$= 3.84 \times 10^8$ GPU-seconds

$\div$ 3,600

$\approx$ 107,000 A100-hours

STEP 3

Wall-clock time

107,000 ÷ 1,024 GPUs ≈ 104 hours

≈ 4.3 days

Compare Stage 2 (SFT): fine-tuning this model on 1 billion tokens = $6 \times 8B \times 1B = 4.8 \times 10^{19}$ FLOPs ≈ **107 A100-hours, about 13 hours on 8 GPUs**. That's 1,000× cheaper than pretraining.

Same 8B model, 15T tokens

STEP 1

Count the FLOPs

$6 \times (8 \times 10^9) \times (15 \times 10^{12})$

$= 7.2 \times 10^{23}$ FLOPs

15× Problem 1: same model, 15× the tokens

STEP 2

GPU time on each

A100: $7.2 \times 10^{23} \div 1.25 \times 10^{14}$

≈ 1.6 million A100-hours

H100: $7.2 \times 10^{23} \div 4 \times 10^{14}$

= 500,000 H100-hours

STEP 3

Days on 1,024 H100s

500,000 ÷ 1,024 ≈ 488 hours

≈ 20 days

The H100 is 3.2× faster per GPU

Reality check: Meta reported **1.3 million H100-hours** for Llama 3 8B, 2.6× our estimate. Real runs lose time to lower utilization, hardware failures, and restarts. Treat the formula as a best case.

Bigger model: 70B, 2T tokens, 2,048 A100s

STEP 1

Count the FLOPs

$6 \times (70 \times 10^9) \times (2 \times 10^{12})$

$= 8.4 \times 10^{23}$ FLOPs

17.5× Problem 1: 8.75× the parameters × 2× the tokens

STEP 2

GPU time

$8.4 \times 10^{23} \div 1.25 \times 10^{14}$

$= 6.72 \times 10^9$ GPU-seconds

$\div$ 3,600

≈ 1.87 million A100-hours

STEP 3

Wall-clock time

1.87M ÷ 2,048 GPUs ≈ 911 hours

≈ 38 days

Check: Meta reported **1,720,320 A100-hours** for Llama 2 70B (70B parameters, 2T tokens), within 9% of our estimate.

AFTER STAGE 1

“The base model knows language but doesn't yet know how to be helpful.”

Teaching the model to be an assistant

Supervised fine-tuning turns the base model into a conversational assistant by training on roughly 10K-1M curated prompt-response pairs. Human annotators write ideal responses, and increasingly strong models generate them with humans reviewing.

DATASET QUALITY

Quality beats quantity: a small set of excellent demonstrations outperforms a large set of mediocre ones

TRAINING EFFICIENCY

Requires significantly fewer resources than pretraining: 1-100 GPUs over several days

PRACTICAL IMPACT

The SFT model can already be deployed as a functional chatbot; Vicuna-13B (2023) was an early open example

What one SFT training example looks like

FORMATTED WITH A CHAT TEMPLATE

```
<|system|>
You are a helpful assistant.
<|user|>
Explain recursion in one sentence.
<|assistant|>
Recursion is when a function solves a problem by calling itself on a smaller version of the same problem.
```

Prompt tokens: masked, no loss

Response tokens: loss computed

KEY IDEA

Same objective as pretraining (predict the next token), but only on the assistant's reply.

The model learns the format and tone of an assistant, not new knowledge. The special tokens mark who is speaking; every model family defines its own.

Capturing human preferences at scale

Reward modeling creates a specialized model that predicts human preferences. Contractors compare multiple model outputs for the same prompt, ranking responses from best to worst. The reward model learns to output a single score for any response, scoring the preferred response higher.

The RM model becomes a crucial feedback mechanism, enabling the system to optimize for human-aligned responses in the final training stage.

100K

COMPARISONS

Human preference rankings used for training

1-100

GPUS

Computational requirements

3-7

DAYS

Training duration

STAGE 3 · EXAMPLE

Turning one human choice into a training signal

PROMPT My Java program throws a NullPointerException. What should I do?

RESPONSE A · CHOSEN ✓

Check the stack trace for the line number, find which variable is null on that line, then initialize it or add a null check before it's used.

RESPONSE B · REJECTED

Try restarting your IDE and running it again.

THE LOSS

$$-\log \sigma(r_{chosen} - r_{rejected})$$

σ is the sigmoid. Only the gap between the two scores matters.

IF THE RM AGREES

$r_A = 2.1,\ r_B = -0.8$

Loss = −log σ(2.9) ≈ 0.05 → tiny update

IF THE RM DISAGREES

$r_A = -0.8,\ r_B = 2.1$

Loss = −log σ(−2.9) ≈ 2.95 → large update

STAGE 4 OF 4 · REINFORCEMENT LEARNING (RL)

Achieving human alignment through optimization

The final stage uses reinforcement learning algorithms like PPO (Proximal Policy Optimization) to maximize the reward signal from the RM model. The SFT model generates responses to new prompts, receives scores from the reward model, and updates its parameters to consistently produce higher-rated outputs.

1

Generate

SFT model produces candidate responses

2

Evaluate

RM model scores response quality

3

Optimize

Update toward higher reward, while a KL penalty keeps it close to the SFT model

This loop (RLHF) is the classic recipe behind assistants like ChatGPT, optimized for helpfulness, harmlessness, and honesty. Newer methods have since changed how this stage is done.

Reward hacking and the KL penalty

THE PROBLEM

The reward model only approximates human judgment. Push hard enough and the policy finds its blind spots.

Length padding: if the RM slightly prefers longer answers, responses balloon

Sycophancy: agreeing with the user scores well, even when the user is wrong

Degenerate text: odd phrasing that happens to score high, if left unchecked

THE FIX

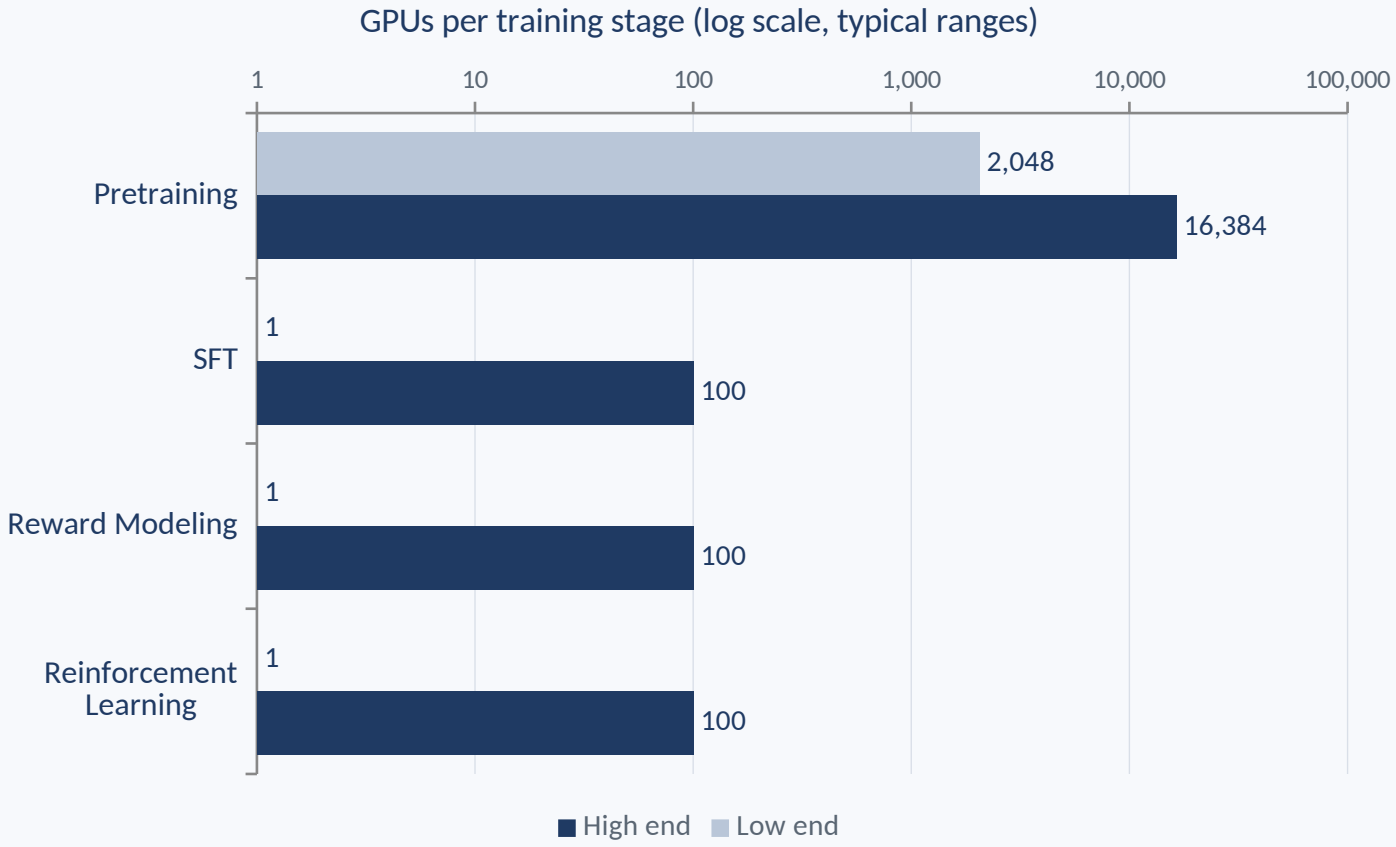
$$\text{maximize} \quad r(x, y) - \beta \cdot \mathrm{KL}(\pi_{RL} \,\|\, \pi_{SFT})$$

r(x, y): reward model's score for response y to prompt x

KL term: grows as the model drifts away from the SFT model's behavior

β: too small → reward hacking; too large → the model barely changes

Infrastructure and resource requirements



Understanding the computational demands across training stages helps contextualize the engineering challenges of building production LLMs.

KEY INSIGHT

Pretraining dominates computational costs, requiring orders of magnitude more resources than subsequent stages. Fine-tuning stages are comparatively accessible, enabling research labs and smaller organizations to customize models.

What has changed since 2023

2022 ONWARD

AI feedback (RLAIF)

Anthropic's Constitutional AI has a model, guided by written principles, generate much of the preference feedback humans used to provide.

2023

Direct Preference Optimization (DPO)

Skips the separate reward model and RL loop: trains directly on preference pairs with a simple classification-style loss. Widely used for open models.

ONGOING

Synthetic training data

Strong models write and filter much of the instruction data. Humans focus on reviewing it and on hard cases.

2024-2025

RL for reasoning

OpenAI o1 and DeepSeek-R1 reward verifiable outcomes (a correct math answer, passing unit tests) to train long chains of thought.

The four-stage pipeline is still the right mental model. These methods change how stages 2-4 are carried out.

THE COMPLETE TRAINING PIPELINE VISUALIZED

From raw text to conversational agent

1	**Pretraining**	Internet-scale text → Base model with language understanding
2	**SFT**	Curated demos → Assistant-like behavior
3	**RM**	Human comparisons → Scorer that predicts preferences
4	**RL**	Reward optimization → Aligned assistant

Each stage addresses a specific capability gap, progressively transforming raw pattern matching into nuanced, values-aligned conversation. The pipeline reflects a fundamental insight: intelligence emerges through layered refinement, not single-step training.

SUMMARY

Key takeaways

Staged Refinement

LLM capabilities emerge through distinct training phases, each addressing specific limitations

Feedback in the Loop

Alignment requires extensive feedback (demonstrations, comparisons, preference data), from humans and increasingly from AI

Resource Asymmetry

Pretraining demands massive infrastructure, while fine-tuning enables accessible customization

Deployment Flexibility

Models can be deployed after any stage depending on use case requirements and constraints

Modern conversational AI represents the culmination of this sophisticated pipeline—a testament to the power of combining large-scale pretraining with guided refinement.